TrustTrace: Tracing Stakeholder Trust to Architecture Decisions in an LLM-Enabled Logistics Compliance Ecosystem

Samarth Annigeri
Independent research portfolio, Montreal, Canada
Samarth.annigeri@mail.mcgill.ca

Technical report, October 2026

Abstract

Large language models (LLMs) are entering systems in which several organisations share data and delegate work to AI services. Trustworthy-AI frameworks name the qualities such systems should have, but give architects little help turning the trust expectations of different stakeholders into requirements, decisions and evidence, especially when those expectations conflict. We present TrustTrace, a design-science artefact set for this translation. It comprises (i) a typed trust metamodel that records who trusts whom, to do what, under which condition and with which evidence; (ii) a step-wise elicitation and analysis method; (iii) a synthetic but realistic multi-party logistics compliance case with 40 trust requirements, 7 explicit conflicts, 15 architecture decision records with structured reflection and 10 reusable patterns; (iv) an executable reference implementation whose traceability tool joins each requirement to its decision, component and passing scenario test; and (v) a pre-registered feasibility pilot of LLM-assisted trust-requirement extraction. In a ten-text pilot with two small local models, Qwen2.5-1.5B reached macro F1 0.73 (naive) and 0.72 (guided); Mistral-7B reached macro F1 0.86 (naive) and 0.91 (guided). The work is a first design cycle. Its validation is analytical and test-based. A practitioner study is designed but not yet run.

1 Introduction

Logistics is a natural setting for LLM assistance. A single cross-border shipment involves a shipper, a freight forwarder, a carrier, a customs broker, a customs authority and, increasingly, platform and model providers. LLMs can suggest tariff classifications, flag inconsistencies between an invoice and a packing list, and answer regulatory questions. Each of these tasks changes who relies on whom. A broker who accepts a suggested classification relies on the assistant, on the passages it retrieved and on the platform operator that configured it. A shipper relies on the platform not to pass its prices to the carrier. An auditor relies on records that nobody could have silently changed.

Trust in this sense is relational and conditional. Organisational research defines it as a willingness to be vulnerable to another party based on expectations about that party [1]. Work on automation stresses that the goal is appropriate reliance, not maximal trust [2]. Jacovi et al. formalise trust in AI as a trustor's belief that a model will uphold an anticipated contract [3]. These views share a structure that architects can use: a trustor, a trustee, an entrusted scope, and conditions under which reliance is warranted.

Existing guidance mostly lists qualities. The European Commission's High-Level Expert Group names seven requirements for trustworthy AI, including human agency and oversight, transparency, privacy and data governance, and accountability [5]. Requirements engineering for AI is an active field [6, 7, 8], and recent work points out what is still missing for responsible AI [9]. Pattern catalogues collect good practice [10]. What remains hard is the step from *these qualities matter* to *this requirement, decided this way, verified by this test*, in an ecosystem where qualities conflict. More transparency can expose confidential data. More automation can reduce human control. Easier provider replacement, which supports sovereignty, can break validated behaviour.

This report asks two research questions, stated in the repository's preregistration:

- **RQ1.** Can a typed trust chain expose missing source, decision, component and passing-scenario evidence in an authored logistics case?
- **RQ2.** Does metamodel-guided prompting improve LLM extraction of atomic trust requirements over a naive prompt?

Contributions. (1) A trust metamodel and analysis method (Section 3). (2) A multi-party logistics case with 40 requirements, 7 conflicts, 15 decision records and 10 patterns (Section 4). (3) An executable reference implementation and traceability gate (Section 5). (4) A pre-registered feasibility pilot of LLM-assisted extraction, with a validated automatic matcher (Section 6). (5) A

protocol for the practitioner study that the artefacts still need (Section 9).

2 Background and related work

Trust and reliance. Mayer et al. separate trust from the trustworthiness factors that justify it [1]. Lee and See argue that designs should support calibrated reliance [2]. Jacovi et al. distinguish warranted from unwarranted trust in AI [3], and Toreini et al. relate trust qualities to concrete machine-learning techniques [4]. We borrow the trustor, trustee and condition structure from this line of work and make it a data model.

Requirements engineering for AI and LLMs. Mapping studies show that requirements work for AI concentrates on data and model qualities, with less attention to multi-stakeholder concerns [6]. Practitioners report difficulty specifying ML quality [7], and the scope of non-functional requirements for ML is still being worked out [8]. LLMs are now studied as assistants for requirements work itself [11, 12]. Our pilot contributes a narrow, reproducible test of one such use. Goal-oriented approaches such as the NFR framework [13] and iStar 2.0 [14] model how design choices help or hurt softgoals. We use a lightweight softgoal graph in that spirit.

Architecture decisions and their human side. Architecture decisions are made by people under uncertainty and bias [15]. Studying them needs social-science methods as well as engineering ones [16]. Reflection influences how designers reason [17], and serious games such as the Ethics-Aware DecidArch game prompt teams to consider stakeholder values [18]. Razavian et al. envision architectural knowledge systems that act as on-demand decision companions [19]. Our decision records add a reflection section motivated by this work. Quality attribute scenarios and ATAM-style analysis [20] supply the six-part scenario form we use for tests.

Engineering AI-enabled systems. AI components behave differently from conventional ones [21]. Test-gated readiness for ML [22], retrieval-augmented generation [23], indirect prompt injection [24], contestability [25], transparency logs [26], scoped authorisation [27] and sovereignty as an architectural objective [28] each inform one of our patterns.

Regulatory context. The case touches the AI Act [29], as amended in 2026 [30], the GDPR [31], the Union Customs Code [32] and the electronic freight transport information (eFTI) regulation [33]. We cite these at the level of the act. Article-level legal text could not be retrieved reliably, and the case makes no legal classification.

3 Method

We follow design science research [34, 35]: problem investigation, treatment design and treatment validation. This report covers one cycle.

Metamodel. The metamodel is a typed Pydantic schema with 16 entity types. A *concern* records provenance (synthetic, documentary or interview). A *trust requirement* names its source concern, trustor, trustee, entrusted scope, condition, fit criterion, priority, and its links to risks, conflicts, tactics, decisions, components and tests. A *quality attribute scenario* gives the six SEI parts: source, stimulus, artefact, environment, response and response measure. *Conflicts* name the requirements in tension and the decision that resolves them. *Evidence* records its provenance (synthetic, measured, assumed, pending). Every link is checked: a dangling identifier, an unresolved conflict or a missing decision document fails validation.

Analysis method. An architect (1) maps stakeholders and trust relationships; (2) elicits concerns from documents, interviews or workshops, keeping provenance; (3) refines each concern into requirements whose fit criterion can fail; (4) identifies conflicts between requirements; (5) writes six-part scenarios; (6) records decisions with considered options, consequences and a reflection on assumptions, dissenting stakeholders and possible bias; and (7) traces each requirement to an executed test. A requirement counts as covered only when a decision, a component, a scenario and a *passing* requirement-tagged test all exist. Coverage is computed from the JUnit report of an actual test run, never from a declared flag.

4 The logistics compliance case

The case has eight authored stakeholder roles: shipper (an SME exporter), freight forwarder, carrier and driver, customs broker, consignee, customs authority or auditor, platform operator and data protection officer. Three LLM-assisted tasks anchor it: tariff classification suggestions for a broker to confirm, cross-document consistency checks, and regulatory question answering with citations. All concerns are *synthetic*: we wrote them from public descriptions of the domain. None comes from interviews.

The 40 requirements comprise a must-priority slice of 16 and 24 should requirements. Table 1 lists the seven conflicts. Each is resolved by a decision record whose consequences section names the residual cost. Fifteen decision records cover the design. Each follows the MADR structure and adds a reflection: the key assumption, which stakeholder would object and why, a plausible bias, and how we checked it. For example, ADR-011 gates raw-document egress by an explicit provider approval flag. Its reflection notes the risk of equating EU location with sovereignty, which is why location and approval are separate attributes.

From the decisions we distil ten patterns (Table 2). Each has context, problem, forces, solution and consequences, plus a pointer to its code and tests in the repository.

Table 1: Conflicts in the case and resolving decisions.

ID	Tension	Decision
C1	Source transparency vs confidential fields	ADR-001
C2	Fast automation vs human authority	ADR-002
C3	Information sharing vs commercial confidentiality	ADR-006
C4	Durable audit evidence vs data minimisation	ADR-008
C5	Contestability vs throughput	ADR-009
C6	Provider replaceability vs validated behaviour	ADR-010
C7	External model capability vs data sovereignty	ADR-011

Table 2: Trust patterns and the requirements whose tests exercise them.

	Pattern	Requirements
P1	Field allowlist at the trust boundary	TR-001, 002, 021
P2	Mandatory human review gate	TR-003, 004, 006
P3	Provenance-carrying answers	TR-009, 010, 025, 030, 035
P4	Anchored tamper-evident log	TR-012, 013, 033
P5	Purpose-bound, expiring delegation	TR-023, 024, 027
P6	Disclosure receipt	TR-039, 040
P7	Contest and override channel	TR-029, 037
P8	Evaluation-gated provider replacement	TR-019, 031, 032
P9	Sovereignty-aware egress gate	TR-036
P10	Output-as-data contract	TR-034

5 Reference implementation and traceability

The implementation is a Python library with a small FastAPI demo. Components are a versioned sharing policy with disclosure receipts, a purpose-bound delegation registry, retrieval with citation checking, a model gateway (egress gate, strict output contract, evaluation-gated replacement), a review queue, contest and override records, explanation cards that separate verbatim quotes from generated interpretation, unit-aware consistency checks, a retention store, and a hash-chained audit log checkpointed to a separate anchor store. Everything is in-process and synthetic. There is no authentication, no real tenant and no customs submission.

Each requirement links to one or more scenario tests marked with its identifier. For example, the test for `TR-034` retrieves a passage that instructs the system to approve an item and checks that the item stays pending and that an output carrying a tool call is rejected. The test for `TR-033` rewrites the whole audit log and checks that verification fails against the stored anchor.

RQ1 result. The traceability tool validates the case with 0 link errors and 0 orphaned decisions, components, tests or conflicts. Of 40 requirements, 38 are covered by a decision, a component, a six-part scenario and a passing requirement-tagged test: all 16 must requirements and 22 of 24 should requirements. The two uncovered requirements (TR-017, TR-018) need verified regulatory sources, which could not be retrieved. These numbers come from an executed test run and are regenerated by `make analyze`.

The answer to RQ1 is a qualified yes. The traceability gate does expose missing evidence: before this cycle it reported 24 should requirements as uncovered, and it still lists the two that need verified legal sources. It cannot tell whether a requirement is the right one, whether a scenario is realistic, or whether a stakeholder would accept the trade-off. Those questions need people.

6 B10 pilot: LLM-assisted trust requirement extraction

Design. The full study, pre-registered on 13 October 2026, plans 24 texts, two model families, five seeds, three prompt conditions and blinded human matching of every prediction. Two of its inputs (verified legal excerpts and an independent retrieval corpus) were not ready, so we ran a reduced feasibility pilot under a dated appendix frozen before any output existed. Ten of the twelve synthetic stakeholder texts were drawn with a fixed seed. The author labelled every atomic obligation in them, with trustor, trustee, scope, condition, fit criterion and an exact evidence span, using a labelling tool that never showed model output. Two local open-weight models, Qwen2.5-1.5B-Instruct [36] and Mistral-7B-Instruct-v0.3 [37], both 4-bit quantised, ran on CPU under two conditions: a naive instruction and a metamodel-guided one. Each text, model and condition was sampled with three seeds, giving 120 generations. A pre-collection check showed that the Mistral chat template silently drops a separate system message, so both models received the instructions inside the user turn.

Matching. Predictions were matched to gold labels automatically by a rule fixed before collection: pair score $= 0.6 \cdot \cos(\text{statements}) + 0.4 \cdot F_1(\text{evidence tokens})$, with BGE-small embeddings [38], a 0.6 threshold and a maximum-weight one-to-one assignment. To test whether the rule agrees with human judgement, the author judged 30 pairs (15 accepted, 15 near-misses), blinded to model, condition and the rule's decision. We report Cohen's κ [39] and treat F1 as unreliable if $\kappa < 0.6$. The guided minus naive difference is paired by text and summarised with a 2000-resample percentile bootstrap over texts [40].

Results. The author labelled 17 atomic obligations across the ten texts. He drafted them first and then entered them in the labelling tool; the entries span 3 minutes, which

Table 3: B10 pilot by model and condition (10 texts, 3 seeds each). F1 is requirement-level, averaged within text then across texts.

Model	Arm	Valid	Macro F1	Prec.	Rec.	Exact ev.
Qwen2.5-1.5B	naive	0.93	0.73	0.92	0.63	0.93
Qwen2.5-1.5B	guided	1.00	0.72	0.92	0.62	0.81
Mistral-7B	naive	0.97	0.86	0.87	0.88	0.94
Mistral-7B	guided	1.00	0.91	0.88	0.95	0.63

is entry time, not total labelling effort. Table 3 gives per-arm results. The guided minus naive difference in macro F1, with 95% bootstrap intervals over texts, was Qwen2.5-1.5B: -0.010 [-0.121, +0.090], guided better on 3 of 10 texts; Mistral-7B: +0.050 [+0.000, +0.117], guided better on 2 of 10 texts. The blinded matcher audit is pending in this build, so F1 values are provisional. Median generation time for the guided arm was Qwen2.5-1.5B 13 s (CPU, 2 threads, cloud container), Mistral-7B 5 s (Metal GPU, M1 Max).

7 Discussion

What the artefacts make possible. The chain turns a vague claim, such as "the platform protects shippers", into a set of checkable statements: which field may reach which role (P1), for which purpose and until when (P5), with what receipt (P6), and what an affected party can do when it goes wrong (P7). The conflict matrix makes the price of each choice explicit. The reflection sections in the decision records make it harder to treat a decision as obvious. For a research programme that studies how stakeholders reason about openness, automation, oversight and accountability, these artefacts are instruments: something to put in front of practitioners and observe how they use, contest or ignore them.

What the pilot suggests. The pilot was designed to answer feasibility questions, and it does. Labelling ten short texts took well under an hour. Small local models produce schema-valid output at the rates in Table 3. The matcher's agreement with blinded human judgement determines whether automatic matching can replace most adjudication in the full study. The guided versus naive contrast is reported with its interval. With ten texts, an interval that crosses zero is the expected outcome, and it should be read as a measure of the effect sizes the full study must be able to detect.

8 Threats to validity

Construct. All concerns are authored. They may encode the author's view of logistics rather than practitioners'. The metamodel's slots may not capture how people actually talk about trust. Requirement F1 measures agreement with one person's labels, not correctness.

Internal. Scenario tests verify behaviour of an in-process prototype under the tested conditions only. The matcher's weights and threshold were assumed. The Mistral model file is a third-party conversion whose source revision we did not verify. Both conditions share the output contract, so the contrast isolates the guidance text only.

External. One synthetic case in one domain. Ten short, single-sentence texts. Two small, quantised models on CPU. Legal text is absent from the pilot, and legal provisions were cited only at the level of the act.

Conclusion. Ten text clusters give wide intervals. We make no significance claims and do not test H1 confirmatorily. There is a single annotator and no inter-annotator agreement.

9 Future work

The next design cycle is empirical. A practitioner study is fully specified in the repository: semi-structured interviews with broker, forwarder, compliance and architecture roles; a workshop in which participants use conflict cards derived from Table 1 to make and justify decisions, inspired by serious-game work on architectural decisions [18]; consent and data-management templates; and a thematic analysis plan. Its questions follow directly from this report. Do practitioners recognise the trustor and trustee slots in their own reasoning? Which conflicts do they see that we missed? Does the reflection section change their decisions or only their documentation? Do disclosure receipts and contest channels matter to SMEs, or only to auditors?

The full B10 study adds verified legal excerpts, a retrieval arm, a second annotator and five seeds, sized from the pilot's observed variance. A four-year agenda would move from this synthetic case to observed architectural decision-making in an LLM-for-logistics consortium, toward a reference architecture whose patterns have been tried, contested and revised with the people who must rely on them.

Reproducibility

Code, data, configuration, raw model outputs, labels, audit decisions and analysis scripts are in the repository. `make test` runs the quality gates and scenario tests. `make analyze` regenerates every table in this report from raw records. The pilot protocol, deviations and model checksums are committed, and the pilot protocol was committed before collection.

AI-assistance statement

This project was built with AI coding assistants under the author's direction. The author set the research direction and scope, chose the vacancy focus, and personally produced the gold labels and the blinded matcher audit. AI assistants wrote most of the code, documentation and this report's first draft, and ran the experiments. The author reviewed the outputs. No institution has reviewed or endorsed this work, and it is not affiliated with TU/e, TNO or the LLM4LM project.

References

[1] R. C. Mayer, J. H. Davis, F. D. Schoorman. An integrative model of organizational trust. *Academy of Management Review*, 20(3), 1995. doi:10.2307/258792

[2] J. D. Lee, K. A. See. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1), 2004. doi:10.1518/hfes.46.1.50.30392

[3] A. Jacovi, A. Marasović, T. Miller, Y. Goldberg. Formalizing trust in artificial intelligence. In *FAccT*, 2021. doi:10.1145/3442188.3445923

[4] E. Toreini et al. The relationship between trust in AI and trustworthy machine learning technologies. In *FAT**, 2020. doi:10.1145/3351095.3372834

[5] High-Level Expert Group on AI. *Ethics Guidelines for Trustworthy AI*. European Commission, 2019. `https://digital-strategy.ec.europa.eu/en/library/ethics-guidelines-trustworthy-ai`

[6] K. Ahmad, M. Abdelrazek, C. Arora, M. Bano, J. Grundy. Requirements engineering for artificial intelligence systems: A systematic mapping study. *Information and Software Technology*, 2023. doi:10.1016/j.infsof.2023.107176

[7] A. Vogelsang, M. Borg. Requirements engineering for machine learning: Perspectives from data scientists. In *REW*, 2019. doi:10.1109/REW.2019.00050

[8] K. M. Habibullah, J. Diaz, G. Gay, J. Horkoff. Scoping of non-functional requirements for machine learning systems. In *RE*, 2024. doi:10.1109/RE59067.2024.00061

[9] D. Zowghi, M. Bano. What's missing in requirements engineering for responsible AI? *IEEE Software*, 2023. doi:10.1109/MS.2023.3302934

[10] Q. Lu, L. Zhu, X. Xu, J. Whittle et al. Responsible AI pattern catalogue: A collection of best practices for AI governance and engineering. *ACM Computing Surveys*, 2024. doi:10.1145/3626234

[11] K. Ronanki, C. Berger, J. Horkoff. Investigating ChatGPT's potential to assist in requirements elicitation processes. In *SEAA*, 2023. doi:10.1109/SEAA60479.2023.00061

[12] C. Arora, J. Grundy, M. Abdelrazek. Advancing requirements engineering through generative AI: Assessing the role of LLMs. In *Generative AI for Effective Software Development*, Springer, 2024. doi:10.1007/978-3-031-55642-5_6

[13] L. Chung, B. A. Nixon, E. Yu, J. Mylopoulos. *Non-Functional Requirements in Software Engineering*. Springer, 2000. doi:10.1007/978-1-4615-5269-7

[14] F. Dalpiaz, X. Franch, J. Horkoff. iStar 2.0 language guide. arXiv:1605.07767, 2016.

[15] A. Tang, M. Razavian, B. Paech, T.-M. Hesse. Human aspects in software architecture decision making: A literature review. In *ICSA*, 2017. doi:10.1109/ICSA.2017.15

[16] M. Razavian, B. Paech, A. Tang. Empirical research for software architecture decision making: An analysis. *Journal of Systems and Software*, 2019. doi:10.1016/j.jss.2018.12.003

[17] M. Razavian, A. Tang, R. Capilla, P. Lago. In two minds: How reflections influence software design thinking. *Journal of Software: Evolution and Process*, 2016. doi:10.1002/smr.1776

[18] R. Alidoosti, P. Lago, E. Poort, M. Razavian. Ethics-aware DecidArch game: Designing a game to reflect on ethical considerations in software architecture design decision making. In *ICSA-C*, 2023. doi:10.1109/ICSA-C57050.2023.00031

[19] M. Razavian, B. Paech, A. Tang. The vision of on-demand architectural knowledge systems as a decision-making companion. *Journal of Systems and Software*, 2023. doi:10.1016/j.jss.2022.111560

[20] R. Kazman, M. Klein, P. Clements. ATAM: Method for architecture evaluation. Technical report CMU/SEI-2000-TR-004, 2000. doi:10.21236/ADA382629

[21] I. Ozkaya. What is really different in engineering AI-enabled systems? *IEEE Software*, 2020. doi:10.1109/MS.2020.2993662

[22] E. Breck, S. Cai, E. Nielsen, M. Salib, D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. In *IEEE Big Data*, 2017. doi:10.1109/BigData.2017.8258038

[23] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020. arXiv:2005.11401

[24] K. Greshake et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *AISec*, 2023. doi:10.1145/3605764.3623985

[25] K. Alfrink, I. Keller, G. Kortuem, N. Doorn. Contestable AI by design: Towards a framework. *Minds and Machines*, 2022. doi:10.1007/s11023-022-09611-z

[26] B. Laurie, A. Langley, E. Kasper. Certificate Transparency. RFC 6962, 2013. doi:10.17487/RFC6962

[27] D. Hardt (ed.). The OAuth 2.0 authorization framework. RFC 6749, 2012. doi:10.17487/RFC6749

[28] M. Esposito, L. Marchesi et al. Sovereign-by-design: A reference architecture for AI and blockchain enabled systems. arXiv:2602.05486, 2026.

[29] Regulation (EU) 2024/1689 (Artificial Intelligence Act). *Official Journal of the European Union*, 2024. `https://eur-lex.europa.eu/eli/reg/2024/1689/oj`

[30] Regulation (EU) 2026/1744 amending Regulation (EU) 2024/1689. *Official Journal of the European Union*, 2026. `https://eur-lex.europa.eu/eli/reg/2026/1744/oj`

[31] Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the European Union*, 2016. `https://eur-lex.europa.eu/eli/reg/2016/679/oj`

[32] Regulation (EU) No 952/2013 laying down the Union Customs Code. *Official Journal of the European Union*, 2013. `https://eur-lex.europa.eu/eli/reg/2013/952/oj`

[33] Regulation (EU) 2020/1056 on electronic freight transport information. *Official Journal of the European Union*, 2020. `https://eur-lex.europa.eu/eli/reg/2020/1056/oj`

[34] A. R. Hevner, S. T. March, J. Park, S. Ram. Design science in information systems research. *MIS Quarterly*, 28(1), 2004. doi:10.2307/25148625

[35] R. J. Wieringa. *Design Science Methodology for Information Systems and Software Engineering*. Springer, 2014. doi:10.1007/978-3-662-43839-8

[36] Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.

[37] A. Q. Jiang et al. Mistral 7B. arXiv:2310.06825, 2023.

[38] S. Xiao, Z. Liu, P. Zhang et al. C-Pack: Packed resources for general Chinese embeddings. arXiv:2309.07597, 2023.

[39] J. Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 1960. doi:10.1177/001316446002000104

[40] B. Efron. Bootstrap methods: Another look at the jackknife. *Annals of Statistics*, 7(1), 1979. doi:10.1214/aos/1176344552